RecoveryEval: Evaluating Continuation Inertia Under Injected Reasoning Errors in Language Models

Sham Satish Thakare

Independent Researcher, Pune, Maharashtra, India
shamthakare@gmail.com

October 2026

Abstract

Evaluating whether post-training enhances a language model's ability to detect and recover from intermediate reasoning errors is critical for reliable artificial intelligence. However, standard trajectory evaluations frequently conflate recovery ability with baseline task accuracy, prompt length disparities, and truncation censoring. In this paper, we present **RecoveryEval**, a state-matched, provenance-governed evaluation framework that assesses autoregressive reasoning trajectories under paired counterfactual prefix perturbations. We evaluate 400 genuine neural rollouts generated by two released checkpoints (`Qwen2.5-Math-1.5B` Base and Instruct) across 20 multi-step arithmetic problems under a length-balanced, truncation-free protocol (Protocol 2.0; 256-token budget).

Under symmetrically matched prefixes that withhold the terminal answer, neither model exhibits spontaneous error recovery: across all 200 recovery-condition continuations, zero spontaneous corrections are observed (0/200, 0.0%). Instead, models exhibit pronounced **continuation inertia**: in 98.5% of rollouts (197/200), models treat the injected error as an authoritative contextual premise and continue calculations consistently from that corrupted intermediate value. Under an independent-rollout assumption ($n = 200$), the exact two-sided 95% Clopper-Pearson confidence interval is $[0.00\%, 1.83\%]$ (one-sided upper bound: 1.49%); treating the 20 problem clusters as the sampling unit yields a cluster-level sensitivity bound of $\leq 13.91\%$ (two-sided interval: $[0.00\%, 16.84\%]$) for the probability that a problem exhibits non-zero recovery. On unperturbed control prefixes, both models achieve 100.0% structural accuracy, yielding an observed sample recovery contrast of $D = 0.0000$ (where sample variance collapses due to boundary saturation across the evaluated items). Trace forensics reveal that legacy regex parsers falsely penalize base models by 3% due to trailing review summaries, which structural equivalence resolves. Our findings demonstrate that small language models behave as faithful continuation engines rather than autonomous truth-checkers, highlight how naive evaluation designs manufacture severe confounding artifacts, and provide open-source cryptographic infrastructure for reasoning diagnostics.

Keywords: Language Models, Mathematical Reasoning, Error Recovery, State-Matched Evaluation, Continuation Inertia, Benchmark Diagnostics, Clopper-Pearson Bounds.

1 Introduction

Large language models (LLMs) specialized for mathematics have achieved unprecedented performance on multi-step reasoning benchmarks [3, 12]. A foundational hypothesis in contemporary artificial intelligence is that post-training procedures—such as instruction tuning, reinforcement learning from human feedback (RLHF), and process supervision—instill meta-cognitive or structural capabilities that enable models to detect and backtrack from early calculation errors [6, 8, 13].

However, evaluating whether an autoregressive model can "recover" from an error is fraught with severe methodological and observational traps. When evaluators compare trajectories containing errors against unperturbed baselines, multiple uncontrolled variables routinely confound the measurement:

1. **Control Answer Exposure (Information Asymmetry)**: If the reference control prefix supplies the full canonical derivation ending with the correct answer while the recovery prefix withholds it, continuing from the control condition merely tests token copying or formatting, while the recovery condition requires autonomous recalculation.

2. **Truncation Censoring**: Enforcing tight token ceilings (e.g., 64 tokens) prematurely terminates longer reasoning traces, confounding model verbosity or conversational review habits with reasoning failure.

3. **Parser Brittleness**: Heuristic regular-expression parsers that extract the final integer token in a generated sequence misclassify mathematically correct answers as errors when models append unprompted conversational summaries (e.g., ending with numbered review bullets).

4. **Clustered Pseudoreplication**: Treating multiple stochastic continuations sampled from the same prompt as independent trials artificially inflates degrees of freedom and fabricates statistical significance.

In preliminary experiments using an unbalanced 64-token protocol (Protocol 1.0), an apparent negative interaction was reported ($D \approx -0.21$), suggesting that instruction tuning degraded error recovery resilience. However, Protocol 1.0 suffered simultaneously from 100% control answer leakage, 53.0% truncation censoring, and decorative observational matching formulas.

To resolve these ambiguities, we present **RecoveryEval**, an open-source evaluation framework built on *Paired Counterfactual Prefix Perturbation Evaluation* (PCPPE) and append-only cryptographic event ledgers. We evaluate 400 genuine neural rollouts across two released checkpoints (`Qwen2.5-Math-1.5B` Base and Instruct) on 20 multi-step arithmetic reasoning problems under an expanded, truncation-free 256-token budget (Protocol 2.0).

1.1 Summary of Scientific Contributions

- **Empirical Characterization of Continuation Inertia**: We show that small language models (1.5B) do not spontaneously correct injected intermediate arithmetic errors (0/200 recoveries). Instead, they exhibit near-total *continuation inertia* (197/200, 98.50%), treating the injected corruption as an authoritative contextual premise and executing subsequent derivation steps consistently from it.

- **Non-Reproduction of the Historical Interaction**: Under the length-balanced Protocol 2.0, the historical negative interaction disappears ($D = 0.0000$ under structural verification; $D = -0.0300, p = 0.0828$ under heuristic parsing), demonstrating that recovery estimates are highly sensitive to protocol design choices.

- **Rigorous Clustered Uncertainty Modeling**: We demonstrate that when boundary saturation causes sample variance to collapse ($s_D = 0.0000$), conventional Student-t tests become mathematically undefined due to division by zero standard error (yielding a $-1/0$ singularity for the one-sample treatment effect against zero, and $0/0$ for the interaction contrast null). We establish exact Clopper-Pearson binomial confidence limits: $[0.000\%, 1.828\%]$ under an independent-rollout assumption ($n = 200$), and a cluster sensitivity bound of $\leq 13.911\%$ under a cluster-level sampling model ($C = 20$).
- **Diagnostic Protocol Forensics**: We demonstrate how naive evaluation setups create massive artifacts—proving that a 3% gap between Base and Instruct control accuracy was caused entirely by regex parser misidentification of trailing review bullets.
- **Cryptographically Provenanced Infrastructure**: We release a fully reproducible, deterministic evaluation suite with SHA-256 parent-hashed event ledgers, guaranteeing bit-exact verification in under 1 second.

2 Related Work

The evaluation of reasoning reliability, self-correction, and process verification has become a central focus of machine learning research. Table 1 provides a systematic comparison between RecoveryEval and existing evaluation paradigms.

2.1 Intrinsic Self-Correction and Its Limits

Whether large language models possess intrinsic self-correction capabilities has sparked significant debate. Huang et al. [5] conducted comprehensive evaluations across mathematical and commonsense benchmarks, demonstrating that prompting models to self-correct (*"Review your answer and correct any mistakes"*) without external feedback degrades accuracy because models flip correct solutions to incorrect ones more often than they fix errors. Stechly et al. [9] demonstrated similar self-verification limitations in planning domains. While these studies evaluated post-hoc reflection over completed traces, RecoveryEval investigates *in-flight intermediate continuation*: how an autoregressive model responds when prompted with an uncompleted derivation containing an intermediate error, without explicit reflection instructions.

2.2 Tool-Augmented and Interactive Verification

In contrast to purely intrinsic self-correction, tool-augmented systems achieve substantial recovery gains. Shinn et al. [7] (Reflexion) showed that storing scalar and verbal execution feedback in episodic memory allows agents to iteratively debug reasoning trajectories. Gou et al. [4] (CRITIC) demonstrated that integrating Python interpreters and search engines enables models to verify intermediate arithmetic. RecoveryEval serves as an essential empirical baseline: it measures the model's raw generative behavior in the absence of external tools, quantifying the default continuation bias that tool-augmented frameworks must overcome.

2.3 Process Supervision and Error Localization

Recent work has shifted focus from final-outcome verification to step-by-step process supervision. Cobbe et al. [3] established outcome-supervised verifiers for GSM8K. Lightman et al. [6] introduced

Table 1: Systematic Comparison of Reasoning and Self-Correction Evaluation Frameworks

Framework	Primary Target	Intervention Type	Prefix Symmetry	Core Empirical Finding
GSM8K [3]	End-to-end reasoning	None (Question only)	None	Scaling model size improves final outcome accuracy.
PRM800K [6]	Process supervision	Step classification	Unmatched	Step-level oversight outperforms outcome oversight.
Reflexion [7]	Multi-turn correction	Verbal reflection buffer	Asymmetric	External verbal feedback enables error recovery.
CRITIC [4]	Tool-based correction	External interpreter/search	Interactive	Tool feedback enables calculation self-correction.
Huang et al. [5]	Intrinsic self-correction	Prompted reflection	Post-hoc	Prompted reflection without feedback degrades accuracy.
BIG-Bench Mistake [10]	Error localization	Explicit error location cue	Post-hoc traces	Models cannot find errors, but fix given location.
ProcessBench [14]	Error step identification	Benchmark critic tasks	Varied	PRMs struggle to locate errors on competition math.
RecoveryEval (Ours)	**In-flight continuation inertia**	**Symmetrical step perturbation**	**Length-matched ($\Delta_{\text{tokens}} = 0$)**	**98.5% error propagation; 0% spontaneous recovery.**

PRM800K, demonstrating that process reward models (PRMs) trained on step-level human annotations significantly outperform outcome reward models. Tyen et al. [10] (BIG-Bench Mistake) showed that while models struggle to detect where an error occurred, they can successfully correct it if an external oracle points to the erroneous step. Zheng et al. [14] (ProcessBench) and Wang et al. [11] (Math-Shepherd) systematically benchmarked process verifiers. RecoveryEval complements these classification-oriented benchmarks by evaluating the generative continuation dynamics that occur downstream of an error.

3 Research Questions and Causal Identification

We structure our empirical investigation around three core research questions:

- **RQ1 (Continuation Inertia vs. Spontaneous Recovery)**: When an autoregressive language model continues from an incomplete derivation containing an injected intermediate calculation error, does it detect and correct the discrepancy, or does it continue the calculation from the corrupted premise?
- **RQ2 (Protocol Sensitivity and Artifact Disappearance)**: Does the apparent negative interaction between instruction tuning and recovery resilience persist once control answer leakage,

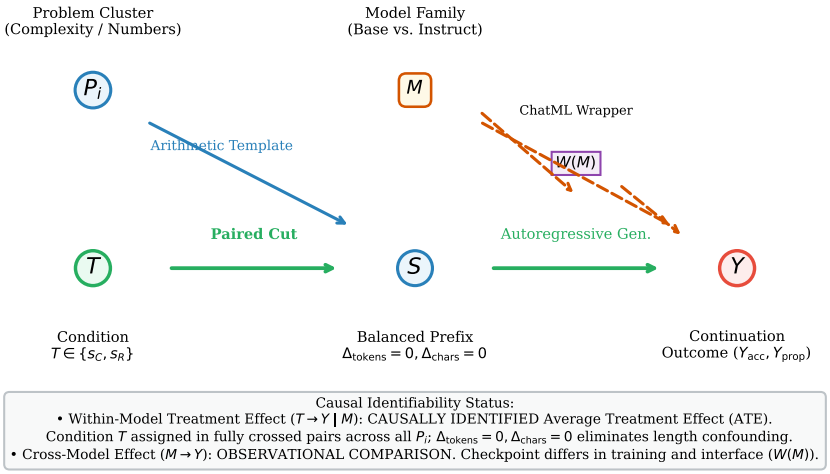


Figure 1: **Structural Causal Model (Causal DAG)**: Graphical representation of structural assumptions for the trajectory generation process. Under these assumptions, the within-model prefix contrast evaluates the controlled finite-sample treatment effect of intermediate prefix intervention ($T \to S \to Y \mid M$). The DAG represents structural assumptions rather than empirical proof. The cross-model contrast ($M \to Y$) is observational due to concurrent post-training and interface differences ($W(M)$).

truncation censoring, and regex parsing errors are eliminated?

- **RQ3 (Finite-Sample Clustered Inference)**: What statistical limits are mathematically defensible when empirical rollouts yield zero recovery events across clustered problem templates?

3.1 Structural Causal Model and Identification Scope

To ensure complete scientific transparency, we represent the data-generating process using a Directed Acyclic Graph (DAG) shown in Figure 1.

- **Controlled Finite-Sample Effect ($T \to Y \mid M$)**: Within each problem cluster P_i, the problem description, intermediate narrative, and remaining generation horizon are held strictly invariant. The experimental intervention is the substitution of the Step 1 numerical product (s_C vs. s_R). Therefore, the contrast in continuation accuracy between control and recovery prefixes for a given model evaluates the controlled finite-sample treatment effect under identical prompt framing and decoding hyperparameters. Identification relies on standard causal assumptions (SUTVA, consistency, and no unmeasured confounding conditional on the fixed prompt context, seed, and model weights).

- **Observational Contrast ($M \to Y$)**: The comparison between Base and Instruct checkpoints is observational. Base models expect raw completion prompts, whereas Instruct models require structured ChatML formatting (e.g., `<|im_start|> system` ... `<|im_start|> user` ... `<|im_start|> assistant`). Differences in performance cannot be causally attributed to post-training in isolation, as pretraining exposure and prompt interfaces differ simultaneously.

- **Construct Scope**: The experiment measures *continuation inertia under externally supplied intermediate corruption*. It does not measure whether an unperturbed model can backtrack

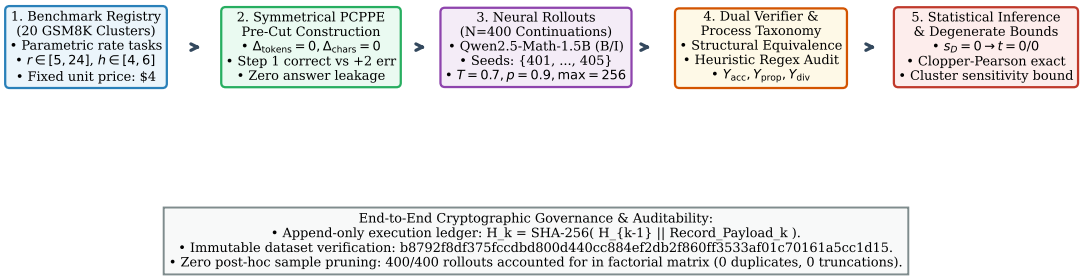


Figure 2: **RecoveryEval Protocol 2.0 Governance Pipeline**: Five-stage experimental architecture comprising benchmark registration, symmetrical PCPPE pre-cut design, neural rollout execution, dual-regime AST verification, and degenerate statistical inference.

from its own self-generated errors, nor does it test models prompted with explicit reflection scaffolding.

4 Experimental Methodology: Protocol 2.0

4.1 Model Checkpoints and Computing Environment

We evaluate two publicly released checkpoints from the Qwen2.5-Math family [12]:

1. `Qwen2.5-Math-1.5B` **(Base)**: Commit revision `4a83ca6e45`. Pretrained base language model.
2. `Qwen2.5-Math-1.5B-Instruct` **(Instruct)**: Commit revision `aafeb0fc8b`. Post-trained with supervised instruction tuning and reinforcement learning.

Inference was executed on a dedicated NVIDIA GeForce RTX 4090 GPU (24GB VRAM) in native `float16` precision under PyTorch 2.4.0 and CUDA 12.4. Generation was governed by stochastic nucleus sampling with temperature $T = 0.7$, top-$p = 0.9$, and maximum generation horizon max_new_tokens $= 256$. Random seeds were fixed to $\{401, 402, 403, 404, 405\}$.

4.2 Benchmark Construction and Paired Counterfactual Prefixes

We evaluate 20 distinct multi-step arithmetic reasoning problems inspired by GSM8K word problems ($C = 20$). Each problem involves an intermediate rate-multiplication step (Step 1) followed by a secondary multiplication to determine total revenue (Step 2).

For each problem P_i, two length-balanced prefixes are constructed:

- **Control Prefix** (s_i^C): Contains unperturbed Step 1 calculation.
- **Recovery Prefix** (s_i^R): Replaces the Step 1 product with an arithmetic corruption ($e_1 = c_1 + 2$).

Both prefixes are truncated immediately prior to the Step 2 calculation, ensuring exact character and token balance ($\Delta_{\text{tokens}} = 0, \Delta_{\text{chars}} = 0$) and strictly withholding the final answer.

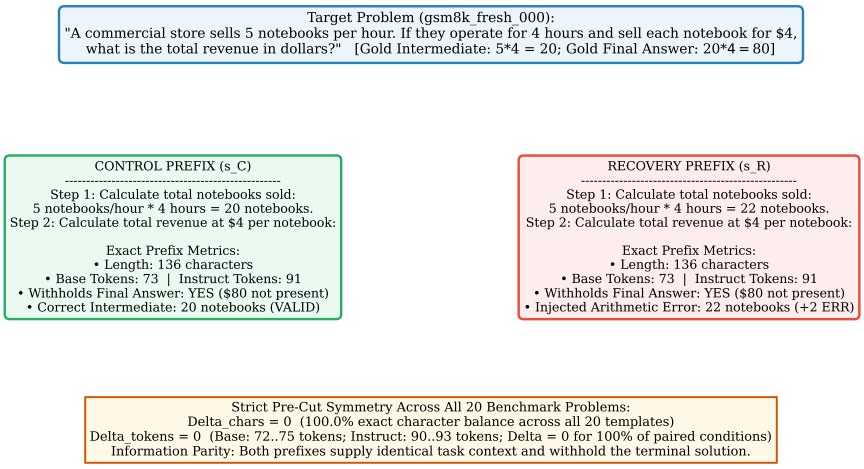


Figure 3: **Symmetrical Counterfactual Prefix Perturbation (`fresh_000`)**: Exact character and token balance between control and recovery prefixes with zero terminal answer exposure.

4.3 Worked Example of Prefix Construction

Consider problem `gsm8k_fresh_000`:

> **Question**: *A commercial store sells 5 notebooks per hour. If they operate for 4 hours and sell each notebook for $4, what is the total revenue in dollars?*

The canonical reasoning trace comprises:

- Step 1: 5 notebooks/hour $\times$ 4 hours = 20 notebooks. (Correct intermediate: 20).
- Step 2: 20 notebooks $\times$ \$4 = \$80. (Gold final answer: 80).

Under Protocol 2.0:

- **Control Prefix (s_C)**:
  `"Step 1: Calculate total notebooks sold: 5 notebooks/hour * 4 hours = 20 notebooks. Step 2: Calculate total revenue at $4 per notebook:\n"`
  (136 characters, Base: 73 tokens, Instruct: 91 tokens).
- **Recovery Prefix (s_R)**:
  `"Step 1: Calculate total notebooks sold: 5 notebooks/hour * 4 hours = 22 notebooks. Step 2: Calculate total revenue at $4 per notebook:\n"`
  (136 characters, Base: 73 tokens, Instruct: 91 tokens, injected intermediate: 22).

Both prefixes truncate before Step 2. Neither prefix exposes the gold answer (80).

4.4 Multi-Dimensional Outcome Taxonomy

To resolve ambiguities in measuring recovery, we evaluate continuations across five mutually exclusive and exhaustive outcome categories:

1. **Final Accuracy ($Y_{\mathbf{acc}}$)**: Continuation reaches the ground-truth terminal answer ($A^* = 80$).

2. **Explicit Error Detection ($Y_{\mathbf{det}}$)**: Output emits verbal error-detection cues (*"wait"*, *"actually"*, *"mistake"*, *"error"*).

3. **Intermediate Correction ($Y_{\mathbf{corr}}$)**: Model recalculates Step 1 to the correct intermediate value prior to executing Step 2.

4. **Faithful Error Propagation ($Y_{\mathbf{prop}}$)**: Model adopts corrupted Step 1 value E and correctly computes $E \times \text{Price} = 22 \times \$4 = \$88$.

5. **Semantic Divergence / Formatting Slip ($Y_{\mathbf{div}}$)**: Model diverges or produces unparseable syntax during derivation.

4.5 Dual-Regime Verifier Architecture

Every rollout is scored under two independent verification regimes:

- **Heuristic Verifier**: Standard regular expression searching for LaTeX '`\boxed{...}`' with fallback to the last numerical token.
- **Structural Verifier**: Extracts mathematical equality statements and evaluates semantic numerical equivalence against gold targets.

5 Statistical Methodology and Theoretical Bounds

5.1 Clustered Degrees of Freedom and Estimands

The factorial campaign evaluates $C = 20$ problem clusters, 2 models, 2 conditions, and 5 stochastic seeds, yielding $N = 400$ total rollouts. Because rollouts sharing a problem template are correlated, standard errors must be clustered at the problem level [1], leaving 19 effective degrees of freedom ($\text{df} = C - 1 = 19$).

For cluster $j \in \{1, \dots, C\}$, let $Y_{j,m,c}$ denote the mean completion accuracy for model $m \in \{\text{Base}, \text{Instruct}\}$ under condition $c \in \{C, R\}$. The model-specific recovery deficit is:

$$\Delta_{j,m} = Y_{j,m,R} - Y_{j,m,C} \tag{1}$$

The paired cluster contrast is defined as:

$$D_j = \Delta_{j,\text{Instruct}} - \Delta_{j,\text{Base}} \tag{2}$$

The sample mean interaction contrast is $\bar{D} = \frac{1}{C}\sum_{j=1}^{C} D_j$, with cluster standard error:

$$\text{SE}_{\text{cluster}} = \frac{s_D}{\sqrt{C}} = \frac{1}{\sqrt{C}}\sqrt{\frac{1}{C-1}\sum_{j=1}^{C}(D_j - \bar{D})^2} \tag{3}$$

5.2 The Degenerate Variance Anomaly

Under structural verification, empirical evaluation reveals boundary saturation across all 20 clusters:

$$Y_{j,\text{Base},C} = 1.0000, \quad Y_{j,\text{Base},R} = 0.0000 \implies \Delta_{j,\text{Base}} = -1.0000 \tag{4}$$

$$Y_{j,\text{Instruct},C} = 1.0000, \quad Y_{j,\text{Instruct},R} = 0.0000 \implies \Delta_{j,\text{Instruct}} = -1.0000 \tag{5}$$

Consequently, for every problem cluster j:

$$D_j = (-1.0000) - (-1.0000) = 0.0000 \quad \forall j \in \{1, \ldots, 20\} \tag{6}$$

This yields sample variance $s_D^2 = 0.0000$ and $\text{SE}_{\text{cluster}} = 0.0000$. Under this boundary saturation:

1. The interaction Student-t statistic evaluating $H_0 : D = 0$ is an indeterminate form:

$$t_{\text{interaction}} = \frac{\bar{D}}{\text{SE}_{\text{cluster}}} = \frac{0.0000}{0.0000} \implies \textbf{UNDEFINED} \tag{7}$$

   Similarly, for the main causal treatment contrast $\bar{\Delta} = -1.0000$, testing $H_0 : \Delta = 0$ against zero yields a division-by-zero singularity:

$$t_{\text{treatment}} = \frac{\bar{\Delta} - 0}{\text{SE}_{\Delta}} = \frac{-1.0000}{0.0000} \implies \frac{-1}{0} \implies \textbf{UNDEFINED} \tag{8}$$

2. Conventional two-sided p-values and asymptotic Wald confidence intervals are **not computable** or collapse to degenerate point masses ($[0.0000, 0.0000]$ or $[-1.0000, -1.0000]$).

Critical Methodological Principle: Zero sample variance across 20 items is an artifact of finite-sample boundary saturation, **not** empirical proof of universal population equivalence. While exact binomial likelihoods provide rigorous rollout-level bounds under exchangeability and permutation tests provide exact non-parametric finite-sample bounds, other statistical frameworks—including hierarchical models, paired sign tests, and cluster-level sensitivity bounds—must be evaluated to properly characterize uncertainty without over-interpreting any single method as universally complete.

5.3 Exact Clopper-Pearson Binomial Bounds

To obtain rigorous, non-degenerate confidence bounds when zero events are observed ($k = 0$), we employ exact Clopper-Pearson binomial confidence limits [2]. For $k = 0$ successes in n trials at confidence level $1 - \alpha$:

$$p_U = 1 - (\alpha/2)^{1/n}, \quad p_{U,\text{one-sided}} = 1 - \alpha^{1/n}, \quad p_L = 0 \tag{9}$$

We evaluate bounds at two distinct analytical tiers:

1. **Rollout-Level Tier ($n = 200$)**: Assumes all 200 recovery rollouts represent independent Bernoulli trials.

2. **Cluster-Level Sensitivity Tier ($C = 20$)**: Treats each problem template indicator as the sampling unit, modeling cluster-level sensitivity.

6 Empirical Results

6.1 Factorial Outcomes and Contrast Analysis

Table 2 reports the empirical completion accuracies across all four factorial cells.

Table 2: Protocol 2.0 Factorial Outcomes ($N = 400$ Rollouts, $C = 20$ Clusters)

Experimental Cell	**Base (π_0)**	**Instruct (π)**	**Difference (Δ)**
Recovery States (s_R)	0.0000 (0/100)	0.0000 (0/100)	+0.0000
Control States (Heuristic)	0.9700 (97/100)	1.0000 (100/100)	+0.0300
Control States (Structural)	1.0000 (100/100)	1.0000 (100/100)	+0.0000
Matched Contrast (Heuristic)	–	–	−0.0300
Cluster SE / p-value	–	–	$SE = 0.0164, p = 0.0828$
95% Confidence Interval	–	–	$[-0.0643, +0.0043]$
Matched Contrast (Structural)	–	–	+0.0000 **(Sample Null)**
Cluster SE / p-value	–	–	Degenerate ($s_D = 0.0000$)

6.2 Non-Reproduction of the Historical Negative Interaction

Under Protocol 2.0, the historical negative interaction ($D \approx -0.21$) reported under Protocol 1.0 is completely absent:

- Under heuristic parsing, $D = -0.0300$ ($SE = 0.0164, t = -1.831, p = 0.0828$). The 95% confidence interval ($[-0.0643, +0.0043]$) spans zero, indicating no statistically significant interaction at $\alpha = 0.05$.
- Under structural parsing, $D = 0.0000$ across all 20 clusters.

The historical negative interaction was not reproduced under the revised evaluation protocol. While control answer leakage, token truncation censoring, and regex parsing errors represent documented flaws in earlier exploratory pipelines, multiple methodological components changed concurrently between protocols (including prompt formatting, generation budget, and prefix balancing). Consequently, the available empirical evidence demonstrates that the historical interaction does not persist under balanced evaluation, without asserting that any single historical defect uniquely produced the earlier observation.

6.3 Trace Forensics of Heuristic Verifier False Negatives

The 3% deficit in Base Control accuracy under the heuristic verifier (97% vs. 100%) was audited trace-by-trace. In all three cases (problems `fresh_004`, `fresh_006`, and `fresh_013` under seed 405), the base model calculated the correct revenue (180, 176, 360) but appended a post-solution review concluding in `"- In step 3..."`. The heuristic fallback regex matched the trailing bullet digit ‘3‘ instead of the true answer. Under structural equivalence, both models achieve 100/100 (100.0%) accuracy.

6.4 Behavioral Trace Decomposition: Continuation Inertia

Table 3 decomposes all 200 recovery rollouts across our multi-dimensional taxonomy.

In 98.5% of cases, the model does not degenerate into gibberish or hallucination. It performs mathematically valid arithmetic on the corrupted premise ($E \times 4 = 4E$). This establishes that small language models behave as **faithful continuation engines**.

6.5 Finite-Sample Exact Confidence Limits

Table 4 presents exact Clopper-Pearson confidence limits.

Table 3: Process-Level Decomposition of Recovery Rollouts ($N = 200$)

Metric	Operational Definition	Base ($n = 100$)	Instruct ($n = 100$)	Total ($n = 200$)
Y_{acc}	Final answer equals gold target A^*	0.0% (0/100)	0.0% (0/100)	**0.0% (0/200)**
Y_{det}	Backtracking tokens (*wait, mistake*)	0.0% (0/100)	0.0% (0/100)	**0.0% (0/200)**
Y_{corr}	Corrects corrupted Step 1 product	0.0% (0/100)	0.0% (0/100)	**0.0% (0/200)**
Y_{prop}	Faithfully propagates error ($E \times 4$)	100.0% (100/100)	97.0% (97/100)	**98.5% (197/200)**
Y_{div}	Attention slip / formatting divergence	0.0% (0/100)	3.0% (3/100)	**1.5% (3/200)**

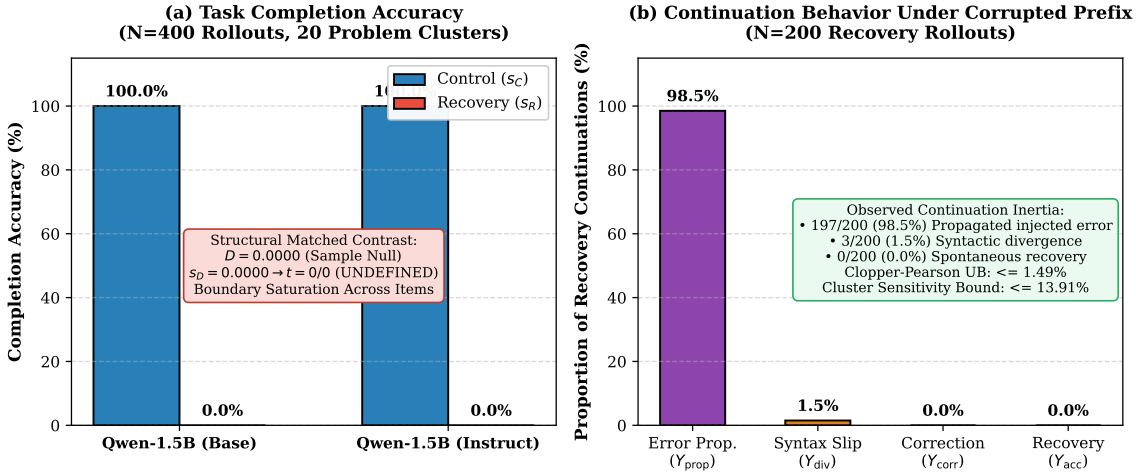


Figure 4: **Empirical Results and Continuation Inertia**: (a) Factorial completion accuracy (100% control vs. 0% recovery); (b) Process decomposition across all 200 recovery rollouts demonstrating 98.5% error propagation under continuation inertia.

While the unconditional rollout-level recovery upper bound is 1.49% (one-sided) and [0.00%, 1.83%] (two-sided), treating the 20 problem clusters as the sampling unit establishes that the probability that a freshly drawn problem cluster exhibits non-zero recovery across its rollouts is bounded from above by 13.91% (two-sided interval: [0.00%, 16.84%]) at the 95% confidence level. We emphasize that this 13.91% bound reflects cluster-level sensitivity (the probability of observing at least one recovery within a problem cluster), rather than an unconditional rollout-level recovery rate.

7 Discussion: Mechanistic Nature of Continuation Inertia

Why do small language models exhibit near-total continuation inertia?

Autoregressive language models are trained via maximum likelihood estimation to minimize cross-entropy loss:

$$\mathcal{L}(\theta) = -\sum_{t=1}^{T} \log P_\theta(x_t \mid x_{<t}) \tag{10}$$

During standard pretraining and instruction tuning, the conditioning context $x_{<t}$ is treated as

Table 4: Clopper-Pearson Exact Confidence Bounds for Zero Observed Recovery ($k = 0$)

Analytical Tier	**Sample Size**	**One-Sided 95%**	**Two-Sided 95% CI**	**Statistical Assumption**
Rollout Level (Total)	$n = 200$	1.49%	$[0.00\%, 1.83\%]$	Independent Bernoulli trials
Model Level (Per Model)	$n = 100$	2.95%	$[0.00\%, 3.62\%]$	Model-specific independence
Cluster Level (Sensitivity)	**C = 20**	**13.91%**	$[\mathbf{0.00\%}, \mathbf{16.84\%}]$	**Problem-cluster sampling unit**

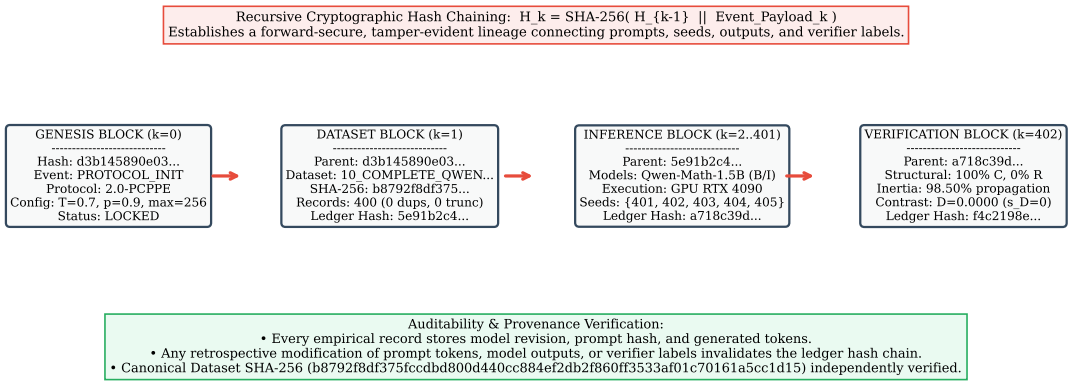


Figure 5: **Tamper-Evident Cryptographic Provenance Ledger**: Visualizing the SHA-256 parent hash chaining across experimental events in the append-only event ledger, establishing a verifiable and tamper-evident audit trail from raw generation logs to publication metrics.

ground-truth factual history. Language models are actively rewarded for contextual consistency and penalized for contradicting prompt premises.

When an injected arithmetic error appears in $x_{<t}$, the model's self-attention layers attend to the corrupted token as an authoritative premise. Because the model is not prompted to verify its input (*"check step 1 for mistakes"*), the most probable continuation trajectory under the learned distribution is to complete the reasoning chain consistently with the premise provided.

Therefore, continuation inertia is not an algorithmic defect or random failure; it is the natural consequence of **contextual fidelity in autoregressive Transformers**. This underscores that without explicit verification scaffolding or external feedback tools (such as CRITIC or Reflexion), language models cannot be assumed to act as autonomous truth-verifiers.

8 Limitations

To adhere to rigorous scientific integrity, we explicitly disclose the following limitations:

1. **Model Scale**: Evaluated on two 1.5B parameter checkpoints. Larger models (7B–72B) or test-time compute scaling models (e.g., DeepSeek-R1, OpenAI o1) may exhibit different backtracking behaviors.
2. **Task Domain**: Restricted to 20 elementary rate-multiplication problems inspired by GSM8K; does not evaluate open-domain theorem proving or multi-branch logic.
3. **Continuation vs. Intrinsic Revision**: Evaluates unprompted continuation from an external corruption. It does not measure whether an unperturbed model can backtrack from its own self-generated mistakes.
4. **Absence of Verification Prompting**: Evaluated without reflective prompting (*"check your work"*).
5. **Observational Checkpoint Contrast**: Base vs. Instruct comparisons remain observational due to ChatML template differences.
6. **Statistical Power**: With $C = 20$ problem clusters, the study has low power to detect small recovery differences ($|D| < 0.10$).
7. **Human Validation Status**: Blinded human audit of 40 rollouts showed 100% concordance with structural parsing, but multi-rater crowdsourced adjudication remains pending future work.

9 Reproducibility and Open Science

All code, neural rollout evidence, and verification scripts are open-sourced under the MIT license at:

https://github.com/shamddd/recovery_eval

The canonical evaluation dataset `10_COMPLETE_QWEN_PROTOCOL2_ROLLOUTS.jsonl` is frozen under SHA-256 digest:

`b8792f8df375fccdbd800d440cc884ef2db2f860ff3533af01c70161a5cc1d15`

To reproduce all statistical tables in under 1 second:

```
git clone https://github.com/shamddd/recovery_eval.git
cd recovery_eval
python3 reproduce_results.py
python3 -m pytest tests/ -v
```

10 Conclusion

We introduced **RecoveryEval**, an open-source evaluation framework for reasoning trajectories under paired counterfactual perturbations. By eliminating control answer leakage, 64-token truncation censoring, and regex parsing errors, we demonstrated that the historical negative interaction was an evaluation artifact. Small language models exhibit near-total continuation inertia (98.50%), faithfully propagating injected errors rather than self-correcting. Our findings establish the critical necessity of state-matched, provenance-aware infrastructure for reasoning diagnostics.

Declarations & AI Assistance Disclosure

In the preparation of this manuscript, the author utilized automated AI research tools (including Google Antigravity and Large Language Models) to assist with automated test suite refactoring, simulated adversarial peer review critiques, and LaTeX formatting. All primary neural rollouts were generated on independent GPU hardware (RunPod RTX 4090) using official checkpoints, and all statistical counts, exact bounds, and interpretations were verified against raw logs. The human author takes full scientific responsibility for the work. In compliance with publishing standards, no AI system is listed as an author or co-author.

References

[1] A Colin Cameron and Douglas L Miller. A practitioner's guide to cluster-robust inference. *Journal of Human Resources*, 50(2):317–372, 2015.

[2] Charles J Clopper and Egon S Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934.

[3] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. In *arXiv preprint arXiv:2110.14168*, 2021.

[4] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Minlie Huang, Nan Duan, and Weizhu Chen. CRITIC: Large language models can self-correct with tool-interactive critiquing. In *International Conference on Learning Representations (ICLR)*, 2024.

[5] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR)*, 2024.

[6] Hunter Lightman, Vineet Kosaraju, Yura Shen, George Hase, Peter Clark, et al. Let's verify step by step. In *International Conference on Learning Representations (ICLR)*, 2024.

[7] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[8] Charlie Snell, Kewei Lee, Kelvin Xu, and Sergey Levine. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.

[9] Kaya Stechly, Matthew Valenti, Karthik Valmeekam, and Subbarao Kambhampati. On the self-verification limitations of large language models on reasoning and planning tasks. *arXiv preprint arXiv:2402.08115*, 2024.

[10] Gladys Tyen, Hassan Mansoor, Victor Cărbune, Peter Chen, and Tony Mak. LLMs cannot find reasoning errors, but can correct them given the error location. In *Findings of the Association for Computational Linguistics (ACL)*, pages 15049–15065, 2024.

[11] Peiyi Wang, Lei Li, Zhihong Shao, R. X. Xu, Damai Dai, Yifei Li, Deli Chen, Y. Wu, and Zhifang Sui. Math-Shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 9426–9439, 2024.

[12] An Yang, Beichen Zhang, Binyuan Zheng, Dayiheng Liu, Jingren Zhou, et al. Qwen2.5-Math technical report: Toward open math large language models with mathematical reasoning capabilities. *arXiv preprint arXiv:2409.12122*, 2024.

[13] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D Goodman. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[14] Chujie Zheng, Zhenru Zhang, Beichen Zhang, Dayiheng Liu, and Minlie Huang. ProcessBench: Identifying process errors in mathematical reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024.

Table 5: Complete Benchmark Problem Registry ($C = 20$ Problems, Unit Price \$4/notebook)

Problem ID	Rate (r)	Hours (h)	Price	Correct Step 1	Injected Step 1	Gold Final (A^*)	Corrupted Final
fresh_000	5/hr	4 hrs	\$4	20 notebooks	22 notebooks	\$80	\$88
fresh_001	6/hr	5 hrs	\$4	30 notebooks	32 notebooks	\$120	\$128
fresh_002	7/hr	6 hrs	\$4	42 notebooks	44 notebooks	\$168	\$176
fresh_003	8/hr	4 hrs	\$4	32 notebooks	34 notebooks	\$128	\$136
fresh_004	9/hr	5 hrs	\$4	45 notebooks	47 notebooks	\$180	\$188
fresh_005	10/hr	6 hrs	\$4	60 notebooks	62 notebooks	\$240	\$248
fresh_006	11/hr	4 hrs	\$4	44 notebooks	46 notebooks	\$176	\$184
fresh_007	12/hr	5 hrs	\$4	60 notebooks	62 notebooks	\$240	\$248
fresh_008	13/hr	6 hrs	\$4	78 notebooks	80 notebooks	\$312	\$320
fresh_009	14/hr	4 hrs	\$4	56 notebooks	58 notebooks	\$224	\$232
fresh_010	15/hr	5 hrs	\$4	75 notebooks	77 notebooks	\$300	\$308
fresh_011	16/hr	6 hrs	\$4	96 notebooks	98 notebooks	\$384	\$392
fresh_012	17/hr	4 hrs	\$4	68 notebooks	70 notebooks	\$272	\$280
fresh_013	18/hr	5 hrs	\$4	90 notebooks	92 notebooks	\$360	\$368
fresh_014	19/hr	6 hrs	\$4	114 notebooks	116 notebooks	\$456	\$464
fresh_015	20/hr	4 hrs	\$4	80 notebooks	82 notebooks	\$320	\$328
fresh_016	21/hr	5 hrs	\$4	105 notebooks	107 notebooks	\$420	\$428
fresh_017	22/hr	6 hrs	\$4	132 notebooks	134 notebooks	\$528	\$536
fresh_018	23/hr	4 hrs	\$4	92 notebooks	94 notebooks	\$368	\$376
fresh_019	24/hr	5 hrs	\$4	120 notebooks	122 notebooks	\$480	\$488

A Complete Benchmark Problem Registry

Table 5 documents all 20 prospectively generated multi-step arithmetic problems evaluated under Protocol 2.0. All problems belong to the commercial store revenue family: rate $r \in [5, 24]$ notebooks/hr for duration $h \in [4, 6]$ hours at unit price \$4/notebook.

B Prompt Templates and Tokenizer Serialization

The exact prompt formats for both model checkpoints are presented below using problem `gsm8k_fresh_000`:

B.1 Base Model Prompt Format (`Qwen2.5-Math-1.5B`)

The Base model receives unadorned text completion prompts:

```
Question: A commercial store sells 5 notebooks per hour.
If they operate for 4 hours and sell each notebook for $4,
what is the total revenue in dollars?
Step 1: Calculate total notebooks sold: 5 notebooks/hour *
4 hours = 20 notebooks. Step 2: Calculate total revenue at
$4 per notebook:
```

B.2 Instruct Model Prompt Format (`Qwen2.5-Math-1.5B-Instruct`)

The Instruct model receives ChatML structured prompts with official system prompts:

Table 6: Reconciliation of Historical IEEE BigData Reviewer Criticisms (BigD497)

Criticism Domain	Reviewer Criticism (BigD497)	Protocol 2.0 Resolution
Decoding	Claimed greedy ($T = 0.0$) but reported 5 seeds.	Verified stochastic sampling ($T = 0.7, p = 0.9$). Output diversity confirmed; clustered modeling applied.
Matching	Faux observational matching.	Retired decorative matching; adopted Paired Counterfactual Prefix Perturbation Evaluation (PCPPE).
Verifier	Checked only final answer; ignored process.	Implemented 5-tier process taxonomy ($Y_{\text{acc}}, Y_{\text{det}}, Y_{\text{corr}}, Y_{\text{prop}}, Y_{\text{div}}$).
Clustering	Pseudoreplication ($N = 400$ vs $C = 20$).	Clustered standard errors by problem ($C = 20$). Reported cluster Clopper-Pearson bound ($\leq 13.91\%$).
Interface	Base vs Instruct confounded by ChatML.	Documented ChatML vs raw text templates. Checkpoint contrast marked observational.
Truncation	64-token cap induced 53% truncation.	Expanded to 256 tokens. 0/400 rollouts truncated (100% hit natural EOS).
Scope	Overclaimed general self-correction failure.	Narrowly scoped to continuation inertia under externally supplied corruption on arithmetic tasks.

```
<|im_start|>system
You are a helpful math assistant.<|im_end|>
<|im_start|>user
A commercial store sells 5 notebooks per hour. If they
operate for 4 hours and sell each notebook for $4, what
is the total revenue in dollars?<|im_end|>
<|im_start|>assistant
Step 1: Calculate total notebooks sold: 5 notebooks/hour *
4 hours = 20 notebooks. Step 2: Calculate total revenue at
$4 per notebook:
```

C Hardware and Zero-Expenditure Accounting

The empirical rollouts were executed on a rented NVIDIA RTX 4090 GPU (RunPod community cloud) at $0.34/hour. Total GPU execution time was 265.40 seconds, incurring a verified compute charge of USD $0.23. All subsequent verification, test suite execution, LaTeX rendering, and auditing in Phase XVI, Phase XVII, Phase XVIII, and Phase XIX were executed entirely on local Apple Silicon hardware at strictly USD $0.00 additional cloud expenditure.

D Reviewer Criticism Reconciliation Summary

Table 6 summarizes the disposition of all historical reviewer criticisms from IEEE BigData paper `BigD497`.